CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
DEPARTAMENTO DE COMPUTAÇÃO (DECOM)
Laboratório de Arquitetura e Organização de Computadores I
Prof.ª Poliana Corrêa

PRÁTICA 5

Banco de registradores

Projeto, simulação e implementação em FPGA

Vítor Albuquerque Dinelli

Belo Horizonte, 8 de outubro de 2026

1. Introdução

Todo computador baseado na arquitetura de von Neumann guarda instruções e dados em uma memória, mas a CPU não consegue trabalhar diretamente sobre a memória principal a cada operação, porque isso seria lento demais. Por isso, os dados que estão sendo processados em um determinado momento ficam em registradores: os menores e mais rápidos elementos de memória digital, que ficam dentro do próprio processador e guardam, cada um, uma palavra de bits.

Um banco de registradores é o conjunto desses registradores organizado de modo que cada um possa ser acessado por meio de um endereço. No processador MIPS, por exemplo, o banco possui duas portas de leitura e uma porta de escrita, já que a maior parte das instruções lê dois operandos e grava um resultado. A leitura é assíncrona: o banco se comporta como um multiplexador, e a saída acompanha o endereço sem depender do clock. A escrita é síncrona: o dado só é gravado na borda de subida do clock e apenas quando o sinal de habilitação (*RegWrite*) vale 1.

Nesta prática foi projetado, em Verilog, um banco de registradores parametrizável, com os valores padrão de 8 registradores de 16 bits e a interface definida no roteiro. O projeto foi simulado no ModelSim/Questa para comprovar o comportamento descrito acima e, depois, sintetizado no Quartus Prime e testado na placa FPGA Altera DE10-Lite, usando as chaves como entradas e os displays de 7 segmentos para mostrar, em decimal, os endereços e os dados. Este relatório apresenta cada bloco do projeto, as decisões tomadas e as simulações que evidenciam o funcionamento correto do circuito.

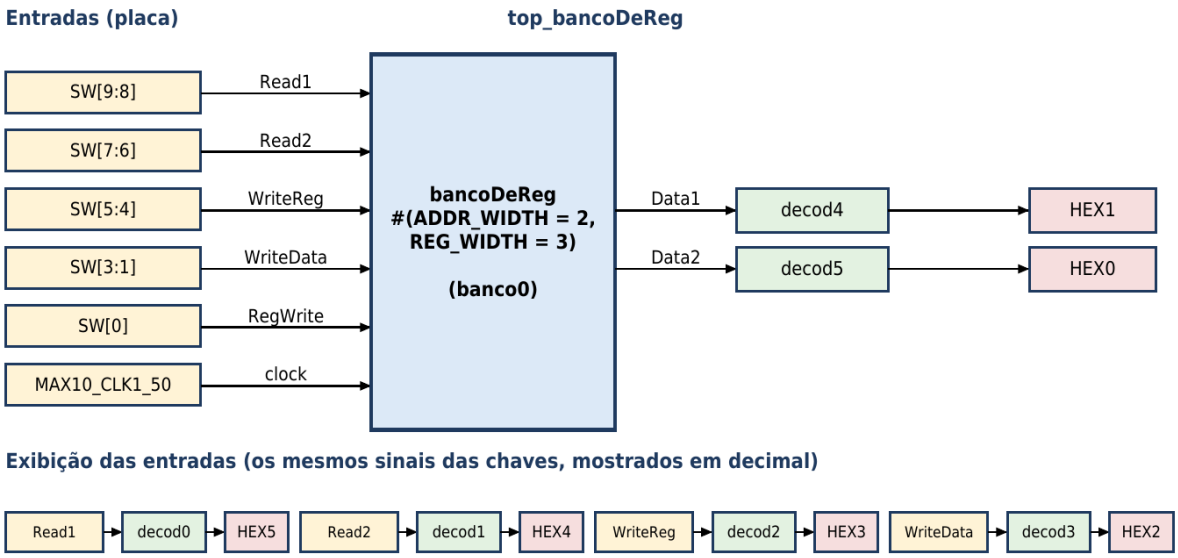
2. Projeto dos blocos

2.1 Visão geral e hierarquia

O projeto é formado por três módulos Verilog. O módulo de nível superior, `top_bancoDeReg`, faz a ligação com os pinos da placa: instancia o banco de registradores e seis decodificadores de 7 segmentos (Figura 1). A Tabela 1 resume os arquivos.

Arquivo	Módulo	Função
`bancoDeReg.v`	bancoDeReg	Banco de registradores parametrizável: duas leituras assíncronas e uma escrita síncrona.
`decodificador.v`	decodificador	Converte um valor binário de 4 bits nos sete sinais de um display de 7 segmentos (ativos em nível baixo).
`top_bancoDeReg.v`	top_bancoDeReg	Nível superior: conecta chaves, clock e displays da placa ao banco e aos decodificadores.

Tabela 1 – Arquivos do projeto.



Os decodificadores convertem o valor binário nos 7 sinais (a–g) de cada display, ativos em nível baixo.

Figura 1 – Diagrama de blocos do módulo de nível superior (configuração usada na placa).

2.2 Módulo bancoDeReg

O módulo segue a interface pedida no roteiro (entradas *Read1*, *Read2*, *WriteReg*, *WriteData*, *RegWrite*, *clock* e saídas *Data1*, *Data2*), com a diferença de que as larguras dos barramentos são definidas por dois parâmetros. A Tabela 2 descreve as portas e os parâmetros, e a Figura 2 mostra a estrutura interna.

Item	Tipo	Largura / valor padrão	Descrição
`ADDR_WIDTH`	parameter	3	Número de bits do endereço (define o número de registradores).
`REG_WIDTH`	parameter	16	Número de bits de cada registrador.
`NUM_REGS`	localparam	1 << ADDR_WIDTH = 8	Quantidade de registradores, calculada a partir de ADDR_WIDTH.
`Read1`, `Read2`	entrada	ADDR_WIDTH	Endereços dos registradores lidos pelas portas 1 e 2.
`WriteReg`	entrada	ADDR_WIDTH	Endereço do registrador que será escrito.
`WriteData`	entrada	REG_WIDTH	Dado a ser gravado.
`RegWrite`	entrada	1	Habilitação da escrita.
`clock`	entrada	1	Relógio; a escrita ocorre na borda de subida.
`Data1`, `Data2`	saída	REG_WIDTH	Conteúdo dos registradores endereçados por Read1 e Read2.

Tabela 2 – Parâmetros e portas do módulo bancoDeReg.

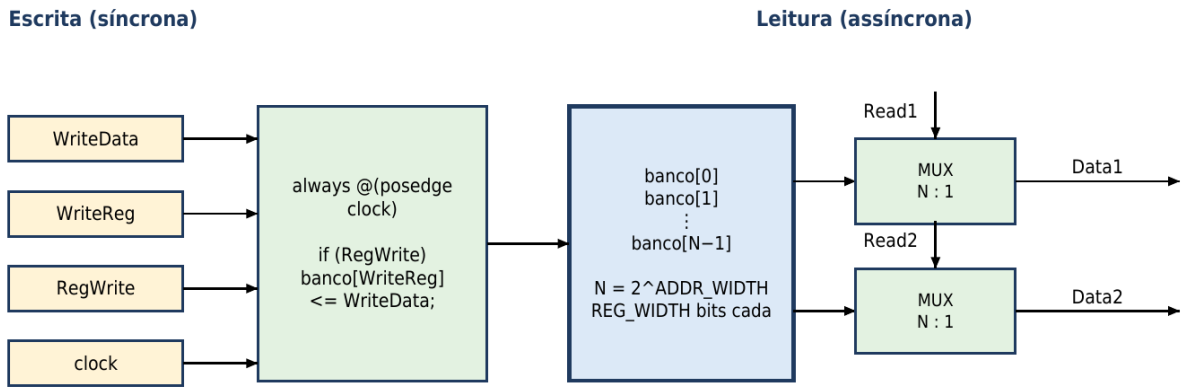


Figura 2 – Estrutura interna do banco de registradores.

As principais decisões de projeto foram:

- **Parametrização.** O número de bits do endereço (*ADDR_WIDTH*) e o da palavra (*REG_WIDTH*) são parâmetros. A quantidade de registradores é derivada deles por um *localparam* (NUM_REGS = 1 << ADDR_WIDTH), de modo que não é possível configurar um valor incoerente. Os valores padrão (3 e 16) correspondem exatamente ao banco pedido no roteiro, de 8 registradores de 16 bits.
- **Armazenamento.** Seguindo a dica do roteiro, os registradores foram descritos como um arranjo do tipo *reg* em vez de flip-flops instanciados um a um. Isso mantém o código curto e faz o tamanho do banco depender apenas dos parâmetros.
- **Leitura assíncrona.** As saídas são atribuições contínuas (*assign*) ao elemento do arranjo indicado pelo endereço. Assim, qualquer mudança em *Read1* ou *Read2* aparece imediatamente em *Data1* ou *Data2*, sem esperar o clock.
- **Escrita síncrona.** A escrita está em um bloco *always @(posedge clock)* e só acontece se *RegWrite* valer 1. Usou-se atribuição não bloqueante (<=), que é a forma adequada para lógica sequencial.
- **Inicialização.** Um bloco *initial* com um laço *for* zera todos os registradores, de modo que o banco começa em um estado conhecido, tanto na simulação quanto na placa.
- **Leitura e escrita no mesmo registrador.** Não há desvio de dados (*forwarding*): se o registrador lido é o mesmo que está sendo escrito, a saída mostra o valor antigo até a borda de subida e o valor novo logo depois dela. Esse é o comportamento de um banco de registradores real, como o do MIPS.
- **Todos os registradores são graváveis.** O roteiro não pede um registrador fixo em zero como o $zero do MIPS, então o registrador 0 funciona como os demais. As duas portas de leitura são independentes e podem ler o mesmo registrador ao mesmo tempo.

O trecho central do módulo é:

```
assign Data1 = banco[Read1];     //leitura assíncrona (porta 1)
assign Data2 = banco[Read2];     //leitura assíncrona (porta 2)

always @(posedge clock) begin   //escrita síncrona
    if (RegWrite)
        banco[WriteReg] <= WriteData;
end
```

2.3 Módulo decodificador (7 segmentos)

Para mostrar os endereços e os dados nos displays de 7 segmentos da placa, foi reaproveitado, sem alterações, o módulo `decodificador` desenvolvido em práticas do semestre anterior em SD.

Ele recebe um valor binário de 4 bits e gera os sete sinais (a a g) de um display, ativos em nível baixo. A escolha se justifica porque o roteiro pede que as entradas e as saídas sejam exibidas em decimal: como os valores usados na placa são pequenos (de 0 a 7), cada um cabe em um único display. Como os valores têm menos de 4 bits, as entradas mais significativas do decodificador são fixadas em `1'b0`, e o mesmo módulo atende tanto os endereços (2 bits) quanto os dados (3 bits). Seu funcionamento interno não é detalhado aqui, e o código está no Apêndice A.

2.4 Módulo de nível superior (top_bancoDeReg)

A placa possui apenas 10 chaves (SW[9:0]). As entradas do banco da placa ocupam exatamente 2 + 2 + 2 + 3 + 1 = 10 bits, e por isso o banco foi instanciado no módulo superior com ADDR_WIDTH = 2 (4 registradores) e REG_WIDTH = 3 (valores de 0 a 7). Isso também faz com que cada endereço e cada dado caiba em um único display, mostrado em decimal, como o roteiro pede. Essa mudança de configuração só foi possível por causa da parametrização: o mesmo módulo, com os valores padrão (3 e 16), é o banco de 8×16 simulado na Seção 3. A Tabela 3 apresenta as ligações.

Sinal	Origem (chaves/clock)	Bits	Display	O que é mostrado
Read1	SW[9:8]	2	HEX5	Endereço lido pela porta 1 (0 a 3)
Read2	SW[7:6]	2	HEX4	Endereço lido pela porta 2 (0 a 3)
WriteReg	SW[5:4]	2	HEX3	Endereço do registrador a escrever (0 a 3)
WriteData	SW[3:1]	3	HEX2	Valor a ser gravado (0 a 7)
RegWrite	SW[0]	1	—	Habilita a escrita
clock	MAX10_CLK1_50 (50 MHz)	1	—	Relógio do banco
Data1	saída do banco	3	HEX1	Conteúdo do registrador Read1
Data2	saída do banco	3	HEX0	Conteúdo do registrador Read2

Tabela 3 – Ligações do módulo de nível superior.

Outras decisões tomadas no módulo de nível superior:

- **Clock.** Foi usado diretamente o clock de 50 MHz da placa. Como ele é muito mais rápido que a mudança manual das chaves, enquanto SW[0] está em 1 o mesmo dado é regravado a cada borda de subida, o que não causa problema, e a gravação passa a valer logo após RegWrite subir.
- **Espelhamento das entradas.** Os displays HEX5 a HEX2 apenas mostram o valor das chaves (endereços e dado a gravar), para que se possa conferir o que está sendo aplicado. Somente HEX1 e HEX0 mostram saídas reais do banco.
- **Leitura imediata.** Como a leitura é assíncrona, HEX1 e HEX0 se atualizam assim que os endereços SW[9:6] mudam, sem que seja necessário gerar um pulso de clock.

3. Simulações

3.1 Configuração

As simulações foram feitas no ModelSim/Questa (Altera Starter FPGA Edition) sobre o módulo `bancoDeReg` com seus parâmetros padrão (ADDR_WIDTH = 3 e REG_WIDTH = 16), isto é, o banco de 8 registradores de 16 bits pedido no roteiro. O clock tem período de 10 ns (bordas de subida em 5 ns, 15 ns, 25 ns, ...) e a simulação dura 400 ns. Nas formas de onda, os endereços (*Read1*, *Read2* e *WriteReg*) estão em binário e os dados (*WriteData*, *Data1* e *Data2*) em decimal.

Além dos sinais do circuito, a janela de ondas mostra o sinal auxiliar *fase*, que não faz parte do circuito: ele foi acrescentado à simulação apenas para identificar qual etapa está em andamento, de 1 a 11. As etapas e os instantes em que ocorrem estão na Tabela 4.

Fase	Intervalo (ns)	O que é verificado
1	12 – 20	Valores iniciais: todos os registradores valem 0 quando lidos pelas duas portas.
2	20 – 196	Escrita de R0 a R7 (10, 200, 3000, 4000, 5000, 60000, 12345 e 54321) e leitura de conferência de cada registrador.
3	196 – 202	Leitura simultânea de dois registradores diferentes: (R0, R7), (R3, R5) e (R6, R1).
4	202 – 206	As duas portas lendo o mesmo registrador: R2 e R7.
5	206 – 237	RegWrite = 0: com WriteReg = R4 e WriteData = 57005, nada pode ser gravado.
6	237 – 249	Leitura assíncrona: os endereços mudam fora de uma borda de clock e as saídas acompanham.
7	249 – 260	Escrita síncrona de 777 em R3: o valor só muda na borda de subida.
8	260 – 280	Leitura e escrita no mesmo registrador (R2 ← 999): valor antigo antes da borda e novo depois.
9	280 – 346	Sobrescrita de R5 (1, 2 e 3) e verificação de que os demais registradores não são alterados.
10	346 – 382	Valores limite: R0 ← 65535 e R7 ← 0.
11	382 – 400	Varredura final de todos os registradores, lidos em pares.

Tabela 4 – Fases da simulação.

Observação sobre os valores negativos. O Questa exibe os dados em decimal com sinal; por isso, nas figuras, valores com o bit mais significativo em 1 aparecem como negativos. São os mesmos valores deste relatório, em complemento de dois: 60000 aparece como −5536, 54321 como −11215, 57005 (0xDEAD) como −8531 e 65535 (0xFFFF) como −1. Nos textos e nas tabelas, os valores são sempre indicados sem sinal.

3.2 Visão geral da simulação

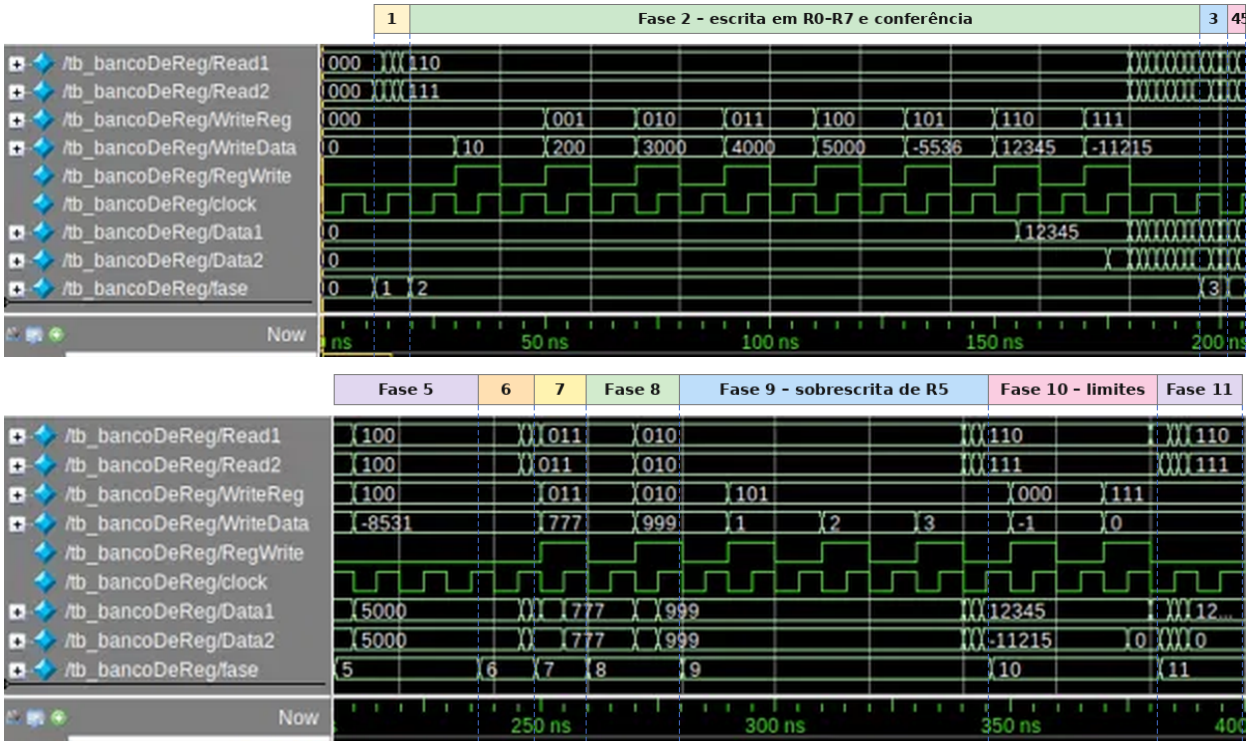


Figura 3 – Simulação completa (0 a 400 ns), dividida em duas partes (0–206 ns e 206–400 ns) para facilitar a leitura. A faixa superior numera as 11 fases e as linhas tracejadas azuis marcam o início e o fim de cada uma.

A Figura 3 mostra a simulação inteira. A faixa colorida no topo indica a fase em andamento, e o sinal *fase*, na última linha, confirma essa numeração. Os pulsos de *RegWrite* aparecem concentrados na fase 2, que grava os oito registradores, e depois em instantes isolados nas fases 7 a 10, em que cada escrita é verificada individualmente.

O objetivo desta figura é dar o mapa da simulação: ela deixa claro que o banco é primeiro preenchido, depois lido de várias formas (duas portas diferentes, mesmo endereço, endereços mudando sem clock) e que, por fim, ele é exercitado nas situações mais delicadas: escrita bloqueada, escrita no mesmo registrador que está sendo lido, sobrescrita e valores limite. As próximas figuras ampliam cada trecho.

3.3 Fases 1 e 2: valores iniciais e escrita em todos os registradores

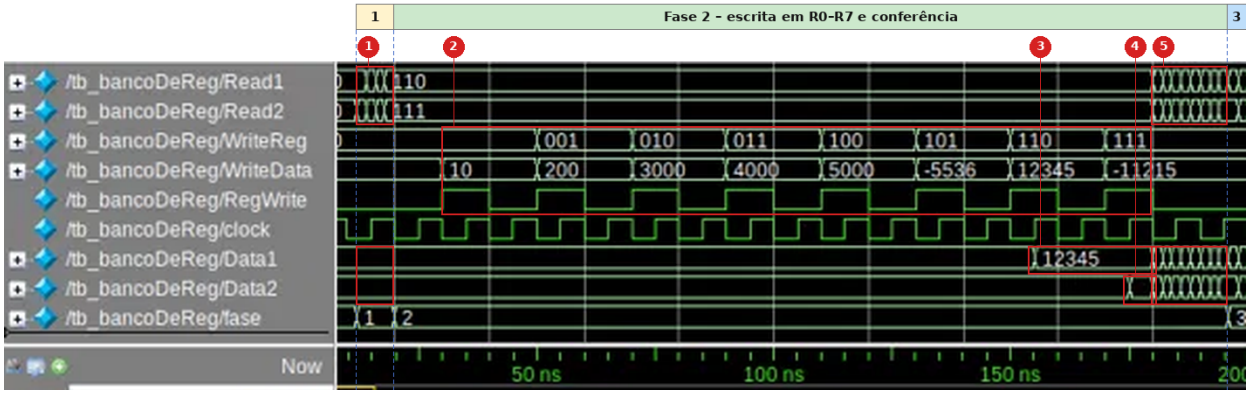


Figura 4 – Fases 1 e 2 (aprox. 8 a 200 ns): inicialização e escrita sequencial de R0 a R7.

[1] Fase 1 (12–20 ns). *Read1* e *Read2* percorrem endereços diferentes, de 2 em 2 ns, mas *Data1* e *Data2* continuam em 0: o banco começa zerado.

[2] Pulsos de RegWrite (30–180 ns). A cada pulso, *WriteReg* avança de 000 a 111 e *WriteData*

recebe 10, 200, 3000, 4000, 5000, 60000 (−5536 na figura), 12345 e 54321 (−11215). Cada pulso cobre uma borda de subida do clock (35, 55, 75, ..., 175 ns), em que o registrador é efetivamente gravado.

[3] Data1 muda para 12345. Durante toda a escrita, *Read1* fica em 110 (R6). A saída só muda depois da borda de subida de 155 ns, quando R6 é gravado.

[4] Data2 muda para 54321. *Read2* fica em 111 (R7) e a saída só muda depois da borda de 175 ns, em que R7 é gravado.

[5] Conferência (180–196 ns). *Read1* e *Read2* varrem os endereços 000 a 111 e as saídas mostram os valores gravados (ampliação na Figura 5).

Esta simulação evidencia dois pontos. Primeiro, o banco inicia com todos os registradores em 0, como foi descrito no bloco *initial*. Segundo, a escrita é de fato síncrona: os itens 3 e 4 mostram que a saída lida só se altera **depois** da borda de subida em que o registrador correspondente é gravado, e não no instante em que *RegWrite* ou *WriteData* mudam.

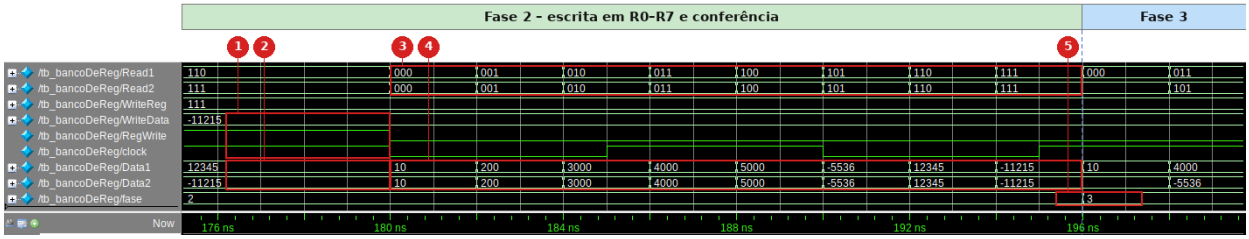


Figura 5 – Fase 2 (aprox. 176 a 198 ns): fim da escrita e leitura de conferência de R0 a R7.

[1] Fim da escrita (até 180 ns). *RegWrite* desce em 180 ns, depois da última gravação (R7, na borda de 175 ns), com *WriteReg* = 111 e *WriteData* = 54321 (−11215).

[2] Últimos registradores gravados. Antes de 180 ns, *Data1* mostra 12345 (R6) e *Data2* mostra 54321, que aparece como −11215 (R7).

[3] Varredura de leitura (180–196 ns). A cada 2 ns, *Read1* e *Read2* avançam juntos de 000 a 111 (R0 a R7), com *RegWrite* em 0 e sem depender do clock.

[4] Valores lidos. *Data1* e *Data2* mostram, nesta ordem, 10, 200, 3000, 4000, 5000, 60000 (−5536), 12345 e 54321 (−11215), exatamente os valores gravados.

[5] Fim da fase 2 (196 ns). O sinal *fase* passa de 2 para 3 e as duas portas passam a ler registradores diferentes.

Nesta ampliação, os valores aparecem dentro das barras, o que permite conferir um a um: cada um dos oito registradores guardou o valor correto, sem interferência entre eles.

3.4 Fases 3, 4 e 5: duas portas de leitura e escrita bloqueada

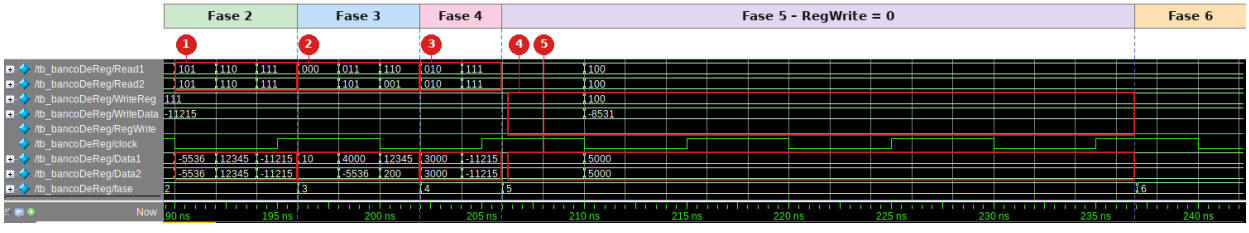


Figura 6 – Fases 3, 4 e 5 (aprox. 190 a 242 ns): leituras simultâneas e RegWrite = 0.

[1] Final da conferência da fase 2 (190–196 ns). As duas portas leem o mesmo endereço (101, 110 e 111) e mostram 60000 (−5536), 12345 e 54321 (−11215).

[2] Fase 3 (196–202 ns). *Read1* e *Read2* apontam para registradores diferentes: (R0, R7), (R3, R5) e (R6, R1). Cada combinação dura apenas 2 ns, por isso os valores só cabem dentro das barras nesta ampliação; eles estão listados na Tabela 5.

[3] Fase 4 (202–206 ns). As duas portas leem o mesmo registrador: (R2, R2) e (R7, R7).

[4] Fase 5 (206–237 ns). *RegWrite* permanece em 0 enquanto *WriteReg* = 100 (R4) e *WriteData* = 57005 (−8531 na figura). Ocorrem três bordas de subida (215, 225 e 235 ns).

[5] Saídas inalteradas. *Data1* e *Data2*, que leem R4, continuam em 5000 durante toda a fase.

Fase	Read1 / Read2	Data1	Data2	Observação
3	000 / 111	10	54321	registradores diferentes, lidos ao mesmo tempo
3	011 / 101	4000	60000	registradores diferentes, lidos ao mesmo tempo
3	110 / 001	12345	200	registradores diferentes, lidos ao mesmo tempo
4	010 / 010	3000	3000	mesmo registrador nas duas portas
4	111 / 111	54321	54321	mesmo registrador nas duas portas

Tabela 5 – Valores lidos nas fases 3 e 4.

Nas fases 3 e 4 queremos mostrar que as duas portas de leitura são independentes: podem apontar para registradores diferentes e devolver, ao mesmo tempo, o conteúdo correto de cada um, ou apontar para o mesmo registrador e devolver o mesmo valor nas duas saídas. A fase 5 evidencia o papel do sinal *RegWrite*: mesmo com um endereço e um dado válidos nas entradas de escrita e com bordas de clock acontecendo, nenhuma gravação ocorre enquanto a habilitação está em 0, e o conteúdo de R4 se mantém em 5000.

3.5 Fases 6, 7 e 8: leitura assíncrona e escrita síncrona

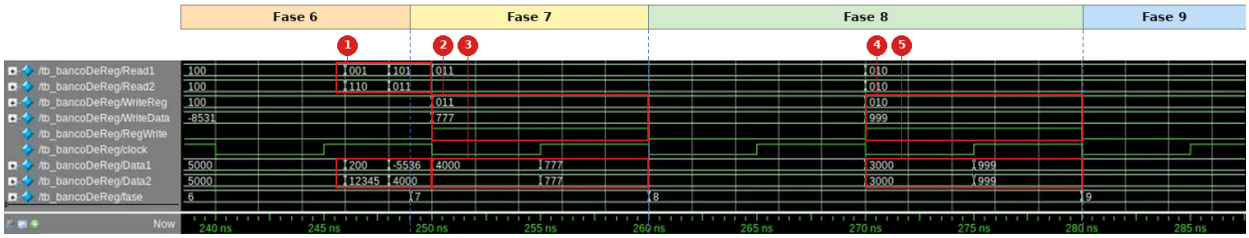


Figura 7 – Fases 6, 7 e 8 (aprox. 238 a 287 ns): leitura assíncrona, escrita na borda e leitura no mesmo registrador escrito.

[1] Fase 6: leitura assíncrona. Logo depois da borda de subida de 245 ns, os endereços mudam em 246 ns para (R1, R6) e em 248 ns para (R5, R3). As saídas mudam no mesmo instante, sem esperar a próxima borda (255 ns): *Data1*/*Data2* passam de 5000/5000 para 200/12345 e depois para 60000/4000 (Tabela 6).

[2] Fase 7: RegWrite sobe. Em 250 ns, *RegWrite* sobe com *WriteReg* = 011 (R3) e *WriteData* = 777, e *Read1* e *Read2* também apontam para R3.

[3] Escrita só na borda. As saídas continuam em 4000 até a borda de subida de 255 ns e só então passam a 777.

[4] Fase 8: mesmo registrador lido e escrito. Em 270 ns, *RegWrite* sobe para escrever 999 em R2, que também está sendo lido por *Read1* e *Read2*.

[5] Valor antigo e valor novo. As saídas mostram 3000 até a borda de 275 ns e 999 depois dela.

Fase 6 – instante	Read1 / Read2	Data1	Data2
246 ns	001 / 110	200	12345
248 ns	101 / 011	60000	4000

Tabela 6 – Valores lidos na fase 6.

A fase 6 mostra que a leitura não depende do clock: a mudança de endereço ocorre 1 ns depois de uma borda de subida, ou seja, longe da seguinte, e mesmo assim a saída acompanha o endereço de imediato, o que caracteriza uma leitura assíncrona (combinacional). As fases 7 e 8 mostram o outro lado: elevar *RegWrite* não altera nada por si só, e o registrador só recebe o novo dado na borda de subida do clock. Como a leitura de R2 e R3 está ativa durante a escrita, vemos também

que o banco devolve o valor antigo até a borda e o novo logo após, sem desvio de dados.

3.6 Fases 9, 10 e 11: sobrescrita, valores limite e conferência final

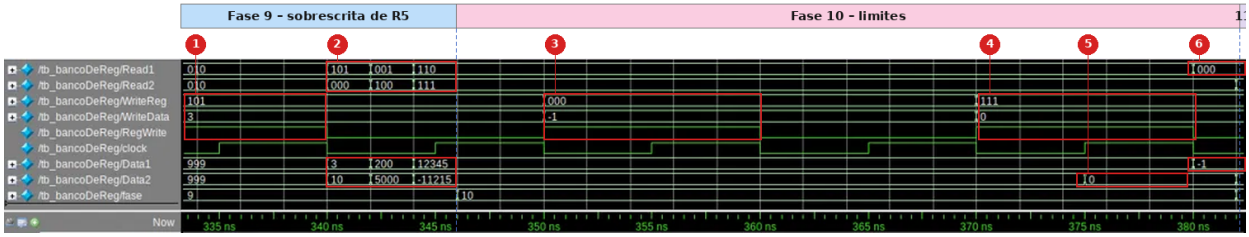


Figura 8 – Fases 9 e 10 (aprox. 333 a 382 ns): fim da sobrescrita de R5 e valores limite.

[1] Fase 9: sobrescrita. Três pulsos de *RegWrite* gravam 1, 2 e 3 em R5 (*WriteReg* = 101), nas bordas de 295, 315 e 335 ns (os dois primeiros aparecem na Figura 3). Nesta figura aparece o último pulso, com *WriteData* = 3. Durante a sequência, *Read1* e *Read2* apontam para R2 e as saídas permanecem em 999.

[2] Conferência (340–346 ns). Leituras em pares mostram R5 = 3 (último valor gravado) e que R0, R1, R4, R6 e R7 continuam com 10, 200, 5000, 12345 e 54321 (−11215).

[3] Fase 10: R0 ← 65535. Em 350 ns, *RegWrite* sobe com *WriteReg* = 000 e *WriteData* = 65535 (0xFFFF, mostrado como −1); a gravação ocorre na borda de 355 ns.

[4] Fase 10: R7 ← 0. Em 370 ns, *RegWrite* sobe com *WriteReg* = 111 e *WriteData* = 0; a gravação ocorre na borda de 375 ns.

[5] Data2 vai a zero. *Data2*, que lê R7, passa de 54321 para 0 depois da borda de 375 ns.

[6] Leitura de (R0, R7). Em 380 ns, *Read1* passa a 000 e *Data1* mostra 65535 (−1), enquanto *Data2* segue em 0.

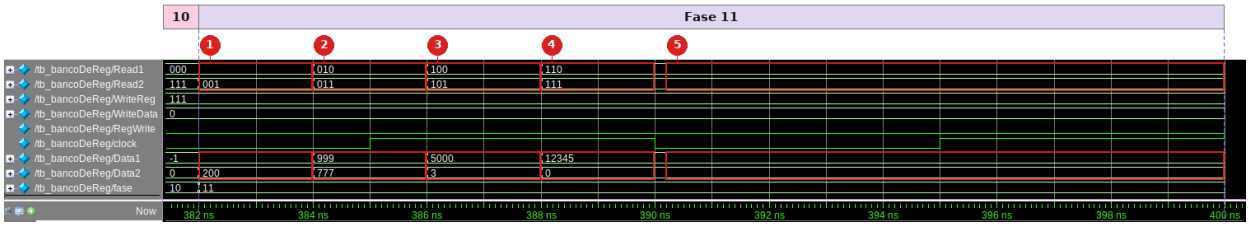


Figura 9 – Fase 11 (382 a 400 ns): varredura final de todos os registradores, lidos em pares.

[1] (R0, R1). De 382 a 384 ns: 65535 (−1) e 200.

[2] (R2, R3). De 384 a 386 ns: 999 e 777.

[3] (R4, R5). De 386 a 388 ns: 5000 e 3.

[4] (R6, R7). De 388 a 390 ns: 12345 e 0.

[5] Fim da simulação. De 390 a 400 ns os endereços e as saídas ficam estáveis, e a simulação termina em 400 ns.

Na fase 9, queremos evidenciar duas coisas: um registrador pode ser sobrescrito várias vezes e guarda sempre o último valor, e escrever em R5 não perturba os demais registradores, o que fica claro pela leitura constante de R2 e pela conferência posterior. A fase 10 testa os extremos da faixa de 16 bits (todos os bits em 1 e todos em 0) e confirma que o registrador 0 pode ser escrito normalmente, já que o projeto não o trata como constante. Por fim, a fase 11 confere o estado final do banco, que concorda com tudo o que foi gravado ao longo da simulação (Tabela 7).

Momento	R0	R1	R2	R3	R4	R5	R6	R7
Início	0	0	0	0	0	0	0	0
Após fase 2	**10**	**200**	**3000**	**4000**	**5000**	**60000**	**12345**	**54321**
Após fase 5	10	200	3000	4000	5000	60000	12345	54321

Momento	R0	R1	R2	R3	R4	R5	R6	R7
Após fase 7	10	200	3000	**777**	5000	60000	12345	54321
Após fase 8	10	200	**999**	777	5000	60000	12345	54321
Após fase 9	10	200	999	777	5000	**3**	12345	54321
Após fase 10 (final)	**65535**	200	999	777	5000	3	12345	**0**

Tabela 7 – Conteúdo dos registradores ao longo da simulação (em negrito, o registrador alterado na fase).

4. Implementação na placa

O projeto foi compilado no Quartus Prime Lite com `top_bancoDeReg` como entidade de nível superior, sintetizado e gravado na placa FPGA, onde foi demonstrado em aula. O uso é direto: para gravar o valor 5 no registrador 2, por exemplo, ajusta-se SW[5:4] = 10 (HEX3 mostra 2) e SW[3:1] = 101 (HEX2 mostra 5) e coloca-se SW[0] em 1. Em seguida, com SW[9:8] = 10 (HEX5 mostra 2), o display HEX1 passa a mostrar 5, e o valor se mantém mesmo depois de SW[0] voltar a 0. Do mesmo modo, SW[7:6] escolhe o registrador mostrado em HEX0. Como a leitura é assíncrona, os displays de saída reagem imediatamente à troca dos endereços.

5. Conclusão

Foi implementado um banco de registradores parametrizável, com valores padrão de 8 registradores de 16 bits, duas portas de leitura assíncronas e uma porta de escrita síncrona controlada por *RegWrite*. As simulações mostraram, em cada uma das fases, que o banco inicia zerado, grava apenas na borda de subida e somente com a escrita habilitada, devolve imediatamente o conteúdo de qualquer registrador quando o endereço de leitura muda e mantém os registradores independentes entre si. A parametrização também permitiu reutilizar o mesmo módulo na placa, com 4 registradores de 3 bits, adaptando o projeto ao número limitado de chaves, e o decodificador de 7 segmentos permitiu conferir os valores em decimal nos displays.

Referências

PATTERSON, D. A.; HENNESSY, J. L. *Computer Organization and Design: the hardware/software interface*. 5. ed. Morgan Kaufmann, 2014.

CORRÊA, P. *Prática 5: Banco de registradores*. Roteiro de aula. Laboratório de Arquitetura e Organização de Computadores I, CEFET-MG, 2026.

Apêndice A – Códigos-fonte

bancoDeReg.v

```
module bancoDeReg #(
    parameter ADDR_WIDTH = 3,
    parameter REG_WIDTH = 16
)
(
    input [ADDR_WIDTH-1:0] Read1, Read2, WriteReg,
    input [REG_WIDTH-1:0] WriteData,
    input RegWrite, clock,
    output [REG_WIDTH-1:0] Data1, Data2
);
    localparam NUM_REGS = 1 << ADDR_WIDTH;

    reg [REG_WIDTH-1:0] banco[0:NUM_REGS -1]; //array de registradores

    //seta as saidas pelo valor dos regs correspondentes
    assign Data1 = banco[Read1]; //leitura assíncrona (porta 1)
    assign Data2 = banco[Read2]; //leitura assíncrona (porta 2)

    //escreve somente em borda de subida do clock
    always @(posedge clock) begin //escrita síncrona
        if(RegWrite) begin //se RegWrite for 1 atualiza o o valor do reg
            banco[WriteReg] <= WriteData;
        end
    end

    //inicializa todos  regs com 0
    integer i;
    initial begin
        for (i = 0; i < NUM_REGS; i = i + 1) begin
            banco[i] = 0;
        end
    end


endmodule
```

decodificador.v

```
module decodificador(A, B, C, D, sA, sB, sC, sD, sE, sF, sG);
    input A, B, C, D;
    output sA, sB, sC, sD, sE, sF, sG;

    assign sA = (~A & ~B & ~C & D) | (B & ~C & ~D);
    assign sB = (B & C & ~D) | (B & ~C & D);
    assign sC = (~B & C & ~D);
    assign sD = (B & C & D) | (B & ~C & ~D) | (~A & ~B & ~C & D);
    assign sE = (D) | (~A & B & ~C);
    assign sF = (C & D) | (~A & ~B & C) | (~A & ~B & D);
    assign sG = (B & C & D) | (~A & ~B & ~C);

endmodule
```

top_bancoDeReg.v

```
module top_bancoDeReg(
    input [9:0] SW,
    input MAX10_CLK1_50,
    output [0:6]HEX0, HEX1, HEX2, HEX3, HEX4, HEX5
```

```
);
    //entrada do banco de reg
    wire [1:0] Read1, Read2, WriteReg;
    wire [2:0] WriteData;
    wire RegWrite;

    assign Read1 = SW[9:8];
    assign Read2 = SW[7:6];
    assign WriteReg = SW[5:4];
    assign WriteData =  SW[3:1];
    assign RegWrite = SW[0];

    //saídas do banco
    wire [2:0] Data1, Data2;

    //escrever na tela os endereço dos regs usados
    decodificador decod0(1'b0, 1'b0, Read1[1], Read1[0], HEX5[0], HEX5[1], HEX5[2], HEX5[3],
HEX5[4], HEX5[5], HEX5[6]);
    decodificador decod1(1'b0, 1'b0, Read2[1], Read2[0], HEX4[0], HEX4[1], HEX4[2], HEX4[3],
HEX4[4], HEX4[5], HEX4[6]);
    decodificador decod2(1'b0, 1'b0, WriteReg[1], WriteReg[0], HEX3[0], HEX3[1], HEX3[2],
HEX3[3], HEX3[4], HEX3[5], HEX3[6]);

    //escrever na tela o valor a ser salvo
    decodificador decod3(1'b0, WriteData[2], WriteData[1], WriteData[0], HEX2[0], HEX2[1],
HEX2[2], HEX2[3], HEX2[4], HEX2[5], HEX2[6]);

    bancoDeReg #(
        .ADDR_WIDTH(2),
        .REG_WIDTH(3)
    ) banco0 (
        .Read1(Read1),
        .Read2(Read2),
        .WriteReg(WriteReg),
        .WriteData(WriteData),
        .RegWrite(RegWrite),
        .clock(MAX10_CLK1_50),
        .Data1(Data1),
        .Data2(Data2)

    );

    //escrever na tela o valor dos registradores
    decodificador decod4(1'b0, Data1[2], Data1[1], Data1[0], HEX1[0], HEX1[1], HEX1[2],
HEX1[3], HEX1[4], HEX1[5], HEX1[6]);
    decodificador decod5(1'b0, Data2[2], Data2[1], Data2[0], HEX0[0], HEX0[1], HEX0[2],
HEX0[3], HEX0[4], HEX0[5], HEX0[6]);



endmodule
```